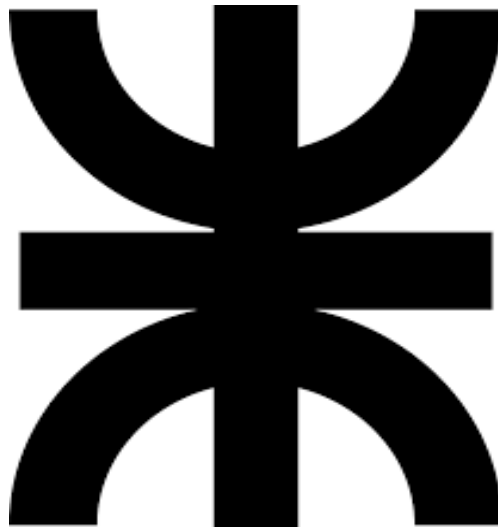


SGBD – Unidad 5 Actividad 2

BITÁCORA – “Dos usuarios (o más) un mismo dato”



**UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RESISTENCIA**

Materia: Base de Datos.

Comisión: K 3.1 **Año:** 2026

Grupo: 7

Integrantes:

- Basualdo Alvarez, Facundo F.
- Godoy Marianela Yasmin.
- Pare, Collante Agustina.
- Sanchez, Maria.
- Sanchez, Romina.
- Toledo, Francisco.

Docentes:

- Prof. Andrés Fantin.
- JTP. Fernández, Juan Carlos.
- Aux, Edgar Nahuel Dominguez.

1. Datos de la corrida

Dato	Valor
Fecha del encuentro	18/09/2026
Motor	PostgreSQL 18.6 sobre Ubuntu Server 26.04 LTS
Servidor	100.116.70.32:5432 — VM del grupo, alcanzable por Tailscale
Base / esquema	sgbd_u5 / sgbd_u5
Tabla de trabajo	cuenta (4 filas, saldos iniciales 10000 / 8000 / 15000 / 5000)
Sesiones	3 por experimento: Sesión A , Sesión B y un puesto de Diagnóstico

Cada sesión se operó **desde una computadora distinta** del grupo, con su propia instalación de pgAdmin, conectadas las tres al mismo servidor. Se usaron las cuentas `alumnoXX` provistas por el setup de la cátedra.

Quién ocupó cada puesto

Cuenta	Integrante	Experimento 1	Experimento 2	Experimento 3
alumno01	facubas	Diagnóstico	—	Sesión B + Diagnóstico
alumno03	agus	Sesión A	—	—
alumno04	fran	—	—	Sesión A
alumno05	romi	Sesión B	Sesión A	—
alumno06	yas	—	Sesión B	—

Parámetros del motor relevantes

```
transaction_isolation
-----
read committed

deadlock_timeout
-----
1s
```

Archivos que acompañan esta bitácora

Archivo	Contenido
<code>sesion-a.sql</code>	Todas las sentencias de la Sesión A, en orden, para los tres experimentos
<code>sesion-b.sql</code>	Todas las sentencias de la Sesión B
<code>diagnostico.sql</code>	Las consultas del puesto de diagnóstico (<code>pg_stat_activity</code> y <code>pg_locks</code>)

Partimos el guion de la cátedra en un archivo por sesión porque la consigna pide entregar los comandos **de cada sesión**, y porque así ninguna de las dos puede ejecutar por error el bloque de la otra.

Sobre las capturas. Son las que sacó cada puesto durante el encuentro. En cinco de ellas (1.1, 1.2, 2.1, 2.3 y 3.4) se colapsó la franja vacía del panel de resultados —la zona sin ninguna fila— para que el texto de la ventana sea legible impreso; el corte se ve como una línea clara. No se editó ningún contenido: las sentencias, los tiempos y las salidas están tal como las devolvió el cliente.

2. Línea temporal de A y B

Las duraciones son las que reportó el motor o el cliente en cada captura. No registramos la hora de reloj de cada paso, así que la línea temporal es la secuencia real de ejecución, que es lo que determina el comportamiento observado.

Experimento 1 — espera por lock

#	Sesión A (alumno03)	Sesión B (alumno05)	Diagnóstico (alumno01)
1	<code>BEGIN; + UPDATE cuenta SET saldo = saldo - 100 WHERE cuenta_id = 1; → responde al instante, transacción abierta</code>		
2	<code>queda idle in transaction, sin confirmar</code>	<code>BEGIN; + UPDATE sobre la misma fila → no retorna, la ventana queda esperando (capturas 1.1 y 1.2: 9,047 s y 1:26,890 sin</code>	

#	Sesión A (alumno03)	Sesión B (alumno05)	Diagnóstico (alumno01)
		completarse)	
3	sigue esperando, sin tocar nada	sigue colgada, sin cancelar	corre pg_stat_activity y pg_locks → capturas 1.3 y 1.4
4	cierra la transacción (ROLLBACK, 151 ms)		
5		el UPDATE se destraba solo en cuanto A cierra; cierra con ROLLBACK (153 ms)	
6			vuelve a correr el diagnóstico → ya no hay ninguna sesión bloqueada (captura 1.6)

Experimento 2 — aislamiento

#	Sesión A (alumno05)	Sesión B (alumno06)
1	BEGIN; + SELECT saldo FROM cuenta WHERE cuenta_id = 3; → 15000.00 (0,863 s)	
2	mantiene la transacción abierta	BEGIN; + UPDATE ... saldo + 1000 WHERE cuenta_id = 3; + COMMIT; → 1 rows affected (15,725 s de corrida total)
3		control posterior al COMMIT: 16000.00
4	segunda lectura dentro de la misma transacción → 16000.00 (0,571 s)	
5	COMMIT;	

Experimento 3 — deadlock controlado

#	Sesión A (alumno04)	Sesión B (alumno01)	Diagnóstico (alumno01)
1	Paso 1A: BEGIN; + UPDATE ... saldo - 10 WHERE cuenta_id = 1; → UPDATE 1 en 587 ms		
2		Paso 1B: BEGIN; + UPDATE ... saldo - 20 WHERE cuenta_id = 2; → UPDATE 1 en 319 ms	
3	Paso 2: UPDATE ... saldo + 20 WHERE cuenta_id = 2; → se cuelga (3,979 s al momento de la captura)	espera sin ejecutar nada	
4	sigue colgada		corre el diagnóstico con el bloqueo en curso → capturas 3.5 y 3.6
5		Paso 3: UPDATE ... saldo + 10 WHERE cuenta_id = 1; → ERROR: deadlock detected (~1 s)	
6	su UPDATE del Paso 2 se completa solo ; queda idle in transaction	queda idle in transaction (aborted)	diagnóstico posterior → capturas 3.8 y 3.9
7	ROLLBACK;	ROLLBACK;	

Que el Paso 3 ocurra **después** del diagnóstico del Paso 4 es deliberado: el ciclo completo dura lo que tarda el motor en detectarlo (≈ 1 s) y no se puede capturar a mano, mientras que la espera del Paso 2 dura indefinidamente y sí se puede documentar con calma.

3. Experimento 1 — ¿Quién bloquea a quién?

La Sesión A abrió una transacción y modificó la cuenta 1 **sin confirmar**. La Sesión B intentó modificar **la misma fila**. Sin cancelar la operación de B, el puesto de diagnóstico investigó la espera desde las vistas del motor. Recién después, A cerró su transacción.

```
-- Sesión A
BEGIN;
SELECT cuenta_id, saldo FROM cuenta WHERE cuenta_id = 1;
UPDATE cuenta SET saldo = saldo - 100 WHERE cuenta_id = 1;
SELECT cuenta_id, saldo FROM cuenta WHERE cuenta_id = 1;

-- Sesión B, mientras A no confirma
BEGIN;
UPDATE cuenta SET saldo = saldo + 50 WHERE cuenta_id = 1;
```

Evidencia de la espera

La ventana de la Sesión B no devuelve resultado: el cliente queda en `Waiting for the query to complete...` y el contador sigue subiendo. B no recibe ningún error.

[CAPTURA — pegar aquí la imagen **e1-01-sesion-bloqueada-alumno05-recorte.png**]

Captura 1.1 — la ventana de `alumno05`, la sesión que el diagnóstico identifica como bloqueada: su `UPDATE` sobre `cuenta_id = 1` lleva 9,047 s sin retornar.

[CAPTURA — pegar aquí la imagen **e1-02-espera-1m26-recorte.png**]

Captura 1.2 — el `UPDATE ... + 50` de `sesion-b.sql` sobre la misma fila: 1 minuto 26 segundos y la sentencia sigue sin completarse. La espera dura exactamente lo que A tarda en cerrar su transacción.

Lo que produce el bloqueo no es el monto sino que **las dos sesiones escriben la misma fila**: en las capturas conviven las dos variantes del `UPDATE` sobre `cuenta_id = 1` porque el experimento se corrió más de una vez.

Sentencia de diagnóstico

Desde una tercera conexión, con la espera en curso:

```
SELECT pid, username, state, wait_event_type, wait_event,
       pg_blocking_pids(pid) AS blocking_pids, query
FROM pg_stat_activity
WHERE datname = current_database()
ORDER BY pid;
```

Quién bloquea a quién

[CAPTURA — pegar aquí la imagen **e1-04-pg-stat-activity-espera.png**]

Captura 1.3 — `pg_stat_activity` con el bloqueo en curso.

Las dos filas que importan, transcritas de esa salida:

```

 pid | username | state | wait_event_type | wait_event |
blocking_pids
-----+-----+-----+-----+-----+-----
25240 | alumno03 | idle in transaction | Client | ClientRead | {}
25246 | alumno05 | active | Lock | transactionid | {25240}

```

Lo que pide la consigna	Lo que devolvió el motor
Sesión bloqueada	PID 25246 (alumno05) — es la única fila con <code>blocking_pids</code> no vacío
Sesión bloqueante	PID 25240 (alumno03) — el PID que aparece dentro de <code>blocking_pids</code>
Estado de espera	<code>state = active</code> , <code>wait_event_type = Lock</code> , <code>wait_event = transactionid</code>
Recurso involucrado	el ID de transacción de la Sesión A, no la tabla (ver <code>pg_locks</code> abajo)

Dos lecturas que conviene dejar anotadas:

- **B no está lenta, está esperando.** Su `state` es `active`, pero `wait_event_type` dice `Lock`. Una consulta lenta estaría activa **sin** evento de espera. El motor distingue las dos cosas.
- **A no está haciendo nada.** Su estado es `idle in transaction`: terminó su sentencia hace rato y dejó la transacción abierta. Toda la espera de B se explica por una sesión que no está trabajando, sólo reteniendo. Sobre esto vuelve la medida preventiva 2.

El recurso en disputa

[CAPTURA — pegar aquí la imagen **e1-05-pg-locks-espera.png**]

Captura 1.4 — pg_locks con el bloqueo en curso.

```

 pid | locktype | relation | mode | granted |
-----+-----+-----+-----+-----+-----
25240 | relation | cuenta_pkey | RowExclusiveLock | true
25240 | relation | cuenta | RowExclusiveLock | true
25240 | virtualxid | [null] | ExclusiveLock | true
25240 | transactionid | [null] | ExclusiveLock | true
25246 | transactionid | [null] | ShareLock | false <-- el lock
pedido y no otorgado
25246 | relation | cuenta | RowExclusiveLock | true
25246 | relation | cuenta | AccessShareLock | true
25246 | tuple | cuenta | ExclusiveLock | true
25246 | virtualxid | [null] | ExclusiveLock | true
25246 | transactionid | [null] | ExclusiveLock | true
25246 | relation | cuenta_pkey | RowExclusiveLock | true
25246 | relation | cuenta_pkey | AccessShareLock | true

```

La única fila con `granted = false` es la de la Sesión B, y su columna `relation` está **vacía**: B **no** está esperando la tabla `cuenta`. Está esperando (`ShareLock` sobre un `transactionid`) a que **termine la transacción de A**. Así implementa PostgreSQL los bloqueos de fila: el lock de fila no se guarda en memoria compartida sino en la propia fila, y el que quiere escribirla se cuelga del ID de transacción del que la escribió primero. Sobre la tabla, en cambio, las dos sesiones tienen su `RowExclusiveLock` **otorgado**: escribir en filas distintas de `cuenta` no habría bloqueado a nadie.

Qué pasó al liberar

En cuanto la Sesión A cerró su transacción, el `UPDATE` de B se completó solo, sin que B tuviera que reintentar nada. Las dos sesiones cerraron con `ROLLBACK`:

[CAPTURA — pegar aquí la imagen `e1-07-rollback.png`]

Captura 1.5 — cierre de las transacciones del experimento 1.

[CAPTURA — pegar aquí la imagen `e1-06-pg-stat-activity-liberada.png`]

Captura 1.6 — el mismo diagnóstico, ya liberada la espera: ninguna fila tiene `blocking_pids` con contenido y no queda ninguna sesión en `Lock`.

Conclusión del experimento 1. PostgreSQL no rechaza ni serializa por error la escritura concurrente sobre la misma fila: hace **esperar** a la segunda sesión hasta que la primera resuelva, y deja esa espera completamente visible en `pg_stat_activity` (quién espera, a quién espera y por qué) y en `pg_locks` (qué lock se pidió y no se otorgó).

4. Experimento 2 — Aislamiento: ¿vemos lo mismo?

Nivel de aislamiento

```
transaction_isolation
-----
read committed
```

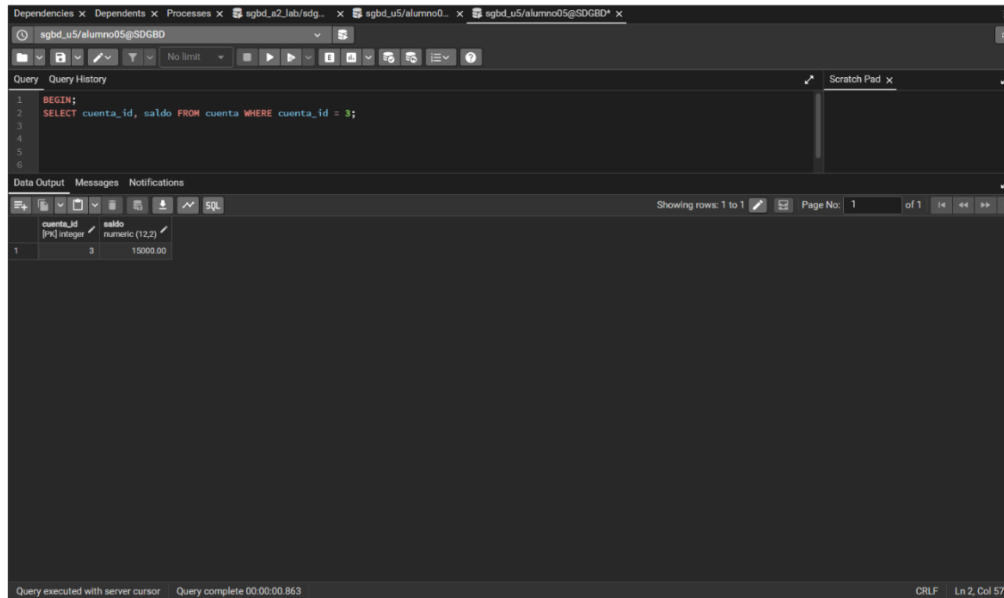
`READ COMMITTED` es el nivel **por defecto** de PostgreSQL y es el que usaron las dos sesiones: no lo cambiamos en ningún momento.

Secuencia y resultado

Sesión A abre transacción y lee el saldo de la cuenta 3:

[CAPTURA — pegar aquí la imagen `e2-01-a-primer-lectura-15000-recorte.png`]

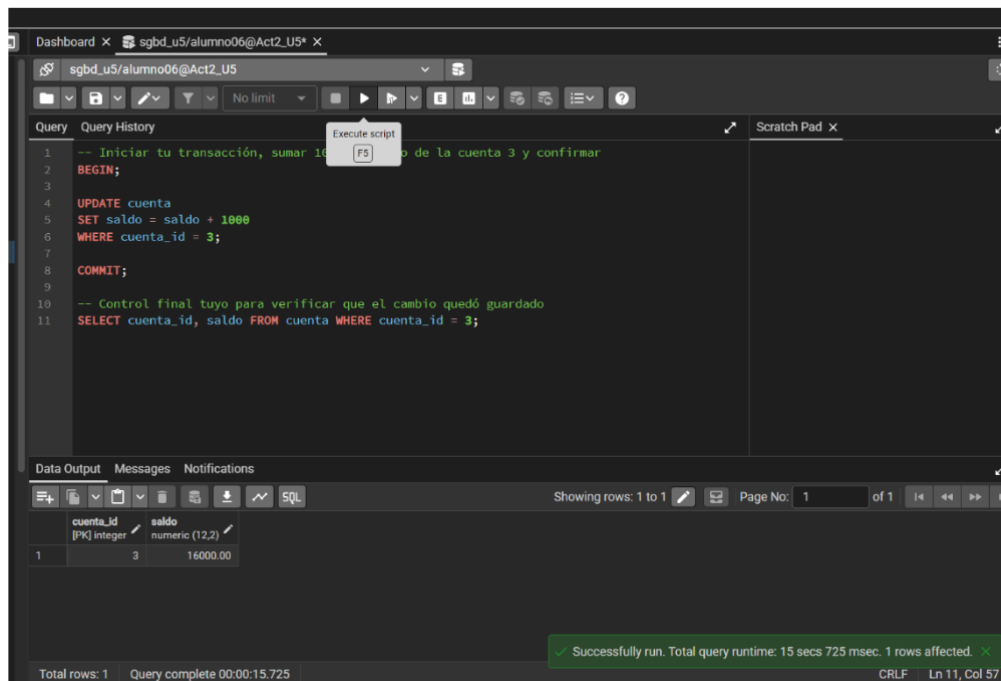
*Captura 2.1 — primera lectura de la Sesión A, dentro de la transacción: **15000.00**.*



Sesión B, en otra computadora, suma 1000 y **confirma**:

[CAPTURA — pegar aquí la imagen **e2-02-b-update-commit-16000.png**]

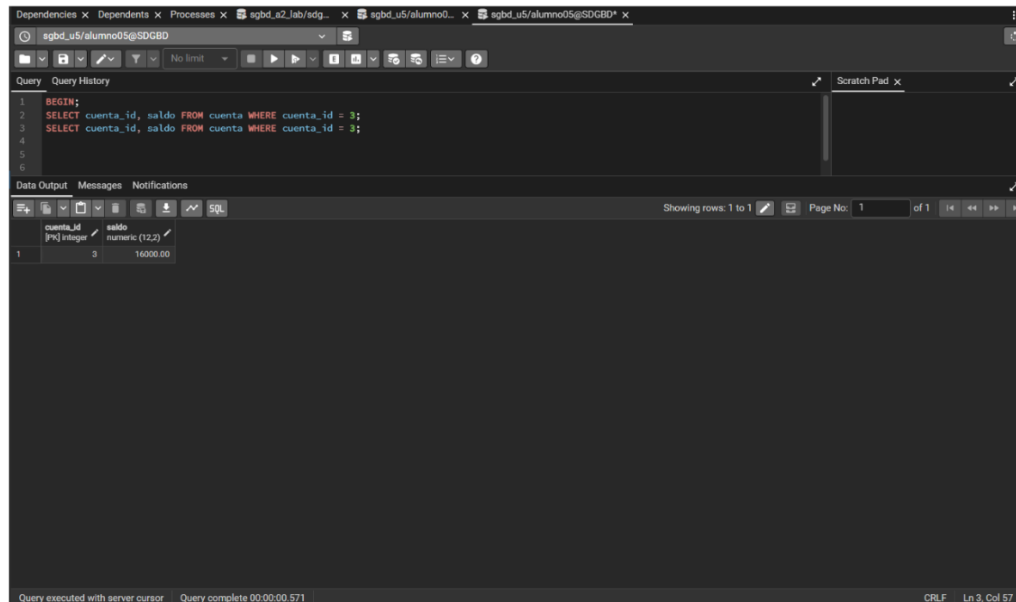
Captura 2.2 — la Sesión B ejecuta el UPDATE, el COMMIT y su control final: 16000.00. El cambio ya está confirmado y es visible para todo el sistema.



Sesión A, sin haber cerrado su transacción, vuelve a leer la misma fila:

[CAPTURA — pegar aquí la imagen **e2-03-a-segunda-lectura-16000-recorte.png**]

Captura 2.3 — segunda lectura de la Sesión A, en la misma transacción: 16000.00.



Resumen:

Momento	Valor leído por A
Primera lectura, antes del COMMIT de B	15000.00
Segunda lectura, misma transacción, después del COMMIT de B	16000.00

5. Experimento 3 — Deadlock controlado

El diseño del ciclo

Cada sesión toma **primero** la fila que la otra va a pedir **después**:

	Primero bloquea	Después pide
Sesión A (alumno04)	cuenta 1	cuenta 2
Sesión B (alumno01)	cuenta 2	cuenta 1

Paso 1 — cada una toma su fila

[CAPTURA — pegar aquí la imagen **e3-01-pasola-sentencia.png** + **e3-02-pasola-resultado.png**]

Capturas 3.1 y 3.2 — Sesión A: *BEGIN* + *UPDATE* sobre la cuenta 1 → *UPDATE* 1 en 587 ms.

[CAPTURA — pegar aquí la imagen **e3-03-paso1b.png**]

Captura 3.3 — Sesión B: *BEGIN + UPDATE sobre la cuenta 2 → UPDATE 1* en 319 ms.

Ninguna de las dos espera: cada una tomó una fila distinta y las dos quedan con la transacción abierta.

Paso 2 — A pide la fila de B y se cuelga

[CAPTURA — pegar aquí la imagen [e3-04-paso2-a-colgada-recorte.png](#)]

Captura 3.4 — la Sesión A intenta la cuenta 2 y queda esperando. Todavía **no** hay deadlock: es una espera común, porque B aún no pidió nada de A.

Diagnóstico con la espera en curso

[CAPTURA — pegar aquí la imagen [e3-05-pg-stat-activity-espera.png](#)]

Captura 3.5 — *pg_stat_activity* con la Sesión A esperando y la B reteniendo.

pid	username	state	wait_event_type	wait_event	blocking_pids
27107	alumno04	active	Lock	transactionid	{27111}
27111	alumno01	idle in transaction	Client	ClientRead	{}

[CAPTURA — pegar aquí la imagen [e3-06-pg-locks-espera.png](#)]

Captura 3.6 — *pg_locks*: la fila *granted = false* es la del PID 27107.

pid	locktype	relation	mode	granted
27107	transactionid	[null]	ShareLock	false
27107	relation	cuenta	RowExclusiveLock	true
27107	relation	cuenta_pkey	RowExclusiveLock	true
27107	tuple	cuenta	ExclusiveLock	true
27107	virtualxid	[null]	ExclusiveLock	true
27107	transactionid	[null]	ExclusiveLock	true
27111	transactionid	[null]	ExclusiveLock	true
27111	relation	cuenta	RowExclusiveLock	true
27111	relation	cuenta_pkey	RowExclusiveLock	true
27111	virtualxid	[null]	ExclusiveLock	true

Hasta acá es el mismo cuadro del experimento 1: **una** sesión esperando a **otra**, en una sola dirección. El motor no tiene nada que resolver; A podría esperar así indefinidamente.

Paso 3 — B cierra el ciclo y el motor aborta una transacción

[CAPTURA — pegar aquí la imagen [e3-07-deadlock-detected.png](#)]

Captura 3.7 — el mensaje que recibió la Sesión B al ejecutar su *UPDATE* sobre la cuenta 1.

```
ERROR: deadlock detected
Process 27111 waits for ShareLock on transaction 2658; blocked by process 27107.
Process 27107 waits for ShareLock on transaction 2659; blocked by process 27111.

SQL state: 40P01
Detail: Process 27111 waits for ShareLock on transaction 2658; blocked by process
27107.
        Process 27107 waits for ShareLock on transaction 2659; blocked by process
27111.
Hint:   See server log for query details.
```

Context: while updating tuple (0,44) in relation "cuenta"

Los dos renglones del `DETAIL` **son** el ciclo, y se leen juntos:

- el proceso **27111** (Sesión B) espera la transacción 2658, que es de **27107**;
- el proceso **27107** (Sesión A) espera la transacción 2659, que es de **27111**.

Cada una espera lo que la otra ya tiene. Ninguna puede avanzar y ninguna va a soltar nada por su cuenta: sin intervención del motor, las dos quedarían detenidas para siempre. Ésa es la diferencia entre este caso y la espera del experimento 1, que se resolvía sola apenas A confirmaba.

Qué transacción fue abortada

La Sesión B (PID 27111, alumno01), es decir **la que cerró el ciclo**, no la que empezó primero. El error le llegó únicamente a ella; la Sesión A ni se enteró: su `UPDATE` del Paso 2, que estaba colgado, se completó solo en cuanto el motor liberó el ciclo.

[CAPTURA — pegar aquí la imagen **e3-08-pg-stat-activity-post-deadlock.png**]

Captura 3.8 — estado inmediatamente posterior.

pid	username	state	blocking_pids
27107	alumno04	idle in transaction	{}
27111	alumno01	idle in transaction (aborted)	{}

La Sesión A quedó `idle in transaction`: viva, con su trabajo hecho y su transacción todavía abierta. La Sesión B quedó `idle in transaction (aborted)`: su transacción fue deshecha y no acepta más sentencias hasta un `ROLLBACK`.

[CAPTURA — pegar aquí la imagen **e3-09-pg-locks-post-deadlock.png**]

Captura 3.9 — `pg_locks` después del deadlock.

pid	locktype	relation	mode	granted
27107	relation	cuenta_pkey	RowExclusiveLock	true
27107	relation	cuenta	RowExclusiveLock	true
27107	virtualxid	[null]	ExclusiveLock	true
27107	transactionid	[null]	ExclusiveLock	true

El PID 27111 **desapareció** de `pg_locks`: al abortar la transacción, el motor liberó de una vez todos sus locks. Y no queda ninguna fila con `granted = false`. Eso es exactamente lo que hace la detección de deadlocks: sacrifica **una** transacción para que el resto del sistema siga.

Por qué esa y no la otra

No es al azar ni es "la más nueva". El parámetro `deadlock_timeout` está en **1 segundo**: el motor no busca ciclos continuamente, sino que **cada sesión que se queda esperando arranca su propio reloj de 1 segundo** y, al vencerlo, revisa si su espera forma un ciclo.

En nuestra corrida, A empezó a esperar en el Paso 2 y su reloj venció enseguida — pero en ese momento B ni siquiera estaba esperando, así que A no encontró ningún ciclo y siguió esperando tranquila (de hecho esperó todo lo que tardamos en sacar las capturas 3.5 y 3.6). Cuando B ejecutó su Paso 3 y venció **su** reloj, el ciclo ya estaba cerrado: **el que lo encuentra es el que se sacrifica**.

De ahí una consecuencia práctica para una aplicación real: la transacción abortada no es

necesariamente la más barata de rehacer, ni la más corta, ni la menos importante. Es simplemente la que llegó última. Por eso conviene evitar el ciclo, que es de lo que tratan las dos medidas de la sección siguiente.

Cierre

Las dos sesiones cerraron con `ROLLBACK`: la abortada porque no aceptaba ninguna otra sentencia, y la sobreviviente para no dejar confirmado un movimiento de prueba. De los tres experimentos, el único que deja un cambio confirmado es el 2 —los +1000 sobre la cuenta 3, que la consigna necesita confirmados para que A pueda verlos—; por eso `sesion-a.sql` termina con un bloque de restauración que devuelve los cuatro saldos a los valores del setup, de modo que el laboratorio se pueda repetir desde el mismo estado.

6. Dos medidas preventivas

Medida 1

Para prevenir los deadlocks proponemos establecer como norma de desarrollo que toda operación que modifique más de una fila las bloquee siempre en el mismo orden, por ejemplo de menor a mayor identificador. Un deadlock no ocurre por falta de recursos ni por cantidad de usuarios, sino porque dos transacciones toman los mismos recursos en secuencias opuestas y cada una queda esperando lo que la otra ya tiene. Si todo el sistema respeta un único orden, esa situación cruzada es imposible: la segunda transacción espera desde su primer paso y se destraba sola cuando la primera confirma. En una transferencia, la aplicación no debería bloquear primero la cuenta de origen y después la de destino, porque el orden quedaría librado a lo que cargó el usuario; debería ordenar los dos identificadores antes de pedir los locks, de modo que dos transferencias simultáneas en sentidos opuestos entre las mismas cuentas se resuelvan una después de la otra.

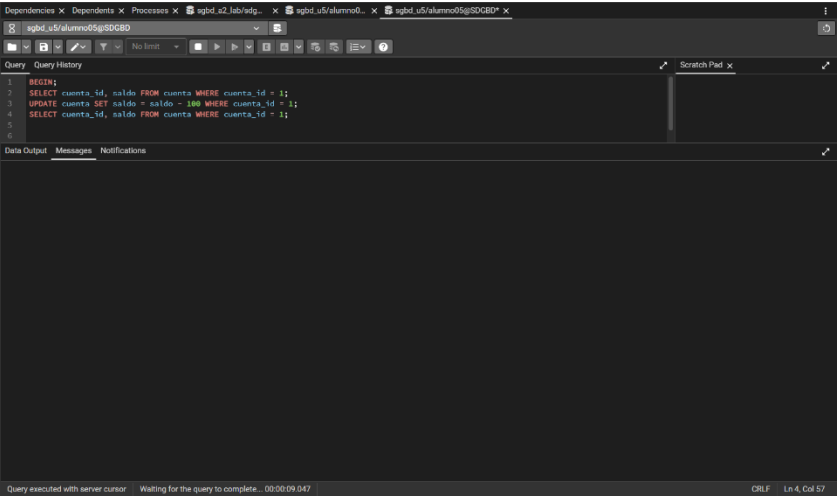
Medida 2

Lo que vimos claramente en el experimento 1 es que, si una transacción se queda abierta mucho tiempo (como pasó con la sesión A), le bloquea totalmente el camino a la sesión B, dejándola colgada y esperando de forma indefinida. Para evitar que un sistema real sufra bloqueos en cadena o cuellos de botella por culpa de esto, un buen enfoque de desarrollo es configurar límites de tiempo de espera automáticos (como `lock_timeout` o `statement_timeout`). Así, si una consulta o transacción tarda más de la cuenta porque el usuario se colgó o hay un problema, el motor la corta sola en lugar de dejar el recurso bloqueado eternamente y trabar a todo el resto de los usuarios.

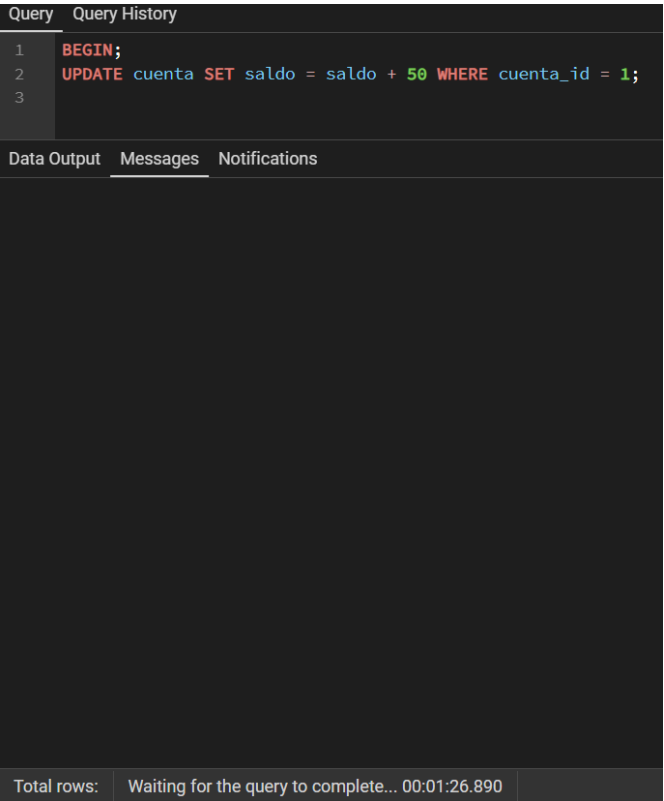
Experimentos:

Experimento 1:

Linea A:



LINEA B:



Query	Query History
1	ROLLBACK;
2	

Data Output	Messages	Notifications
ROLLBACK		
Query returned successfully in 151 msec.		

Query	Query History
1	ROLLBACK;

Data Output	Messages	Notifications
ROLLBACK		
Query returned successfully in 153 msec.		

Captura de Diagnostico:

- Sentencia para el diagnóstico

```
SELECT
    pid,
    username,
    state,
    wait_event_type,
    wait_event,
    pg_blocking_pids(pid) AS blocking_pids,
    query
FROM pg_stat_activity
WHERE datname = current_database()
ORDER BY pid;
```

- Resultado:

Data Output Messages Notifications					
	pid integer	locktype text	relation regclass	mode text	granted boolean
1	25240	relation	cuenta_pk...	RowExclusiveLo...	true
2	25240	relation	cuenta	RowExclusiveLo...	true
3	25240	virtualxid	[null]	ExclusiveLock	true
4	25240	transactio...	[null]	ExclusiveLock	true
5	25242	virtualxid	[null]	ExclusiveLock	true
6	25242	relation	pg_locks	AccessShareLock	true
7	25246	transactio...	[null]	ShareLock	false
8	25246	relation	cuenta	AccessShareLock	true
9	25246	relation	cuenta	RowExclusiveLo...	true
10	25246	virtualxid	[null]	ExclusiveLock	true
11	25246	tuple	cuenta	ExclusiveLock	true
12	25246	transactio...	[null]	ExclusiveLock	true
13	25246	relation	cuenta_pk...	RowExclusiveLo...	true
14	25246	relation	cuenta_pk...	AccessShareLock	true

Data Output								Messages	Notifications
<									

	pid integer	username name	state text	wait_event_type text	wait_event text	blocking_pids integer[]	query text
1	17850	alumno04	idle	Client	ClientRead	{}	SELECT CASE WHEN (SELECT count(extname) FROM pg_catalog.pg_extension WHERE
2	24192	alumno06	idle	Client	ClientRead	{}	/*pga4dash*/ SELECT 'session_stats' AS chart_name, pg_catalog.row_to_json(t) AS chart,
3	25239	alumno03	idle	Client	ClientRead	{}	SELECT pg_cancel_backend(25240);
4	25240	alumno03	idle in transacti...	Client	ClientRead	{}	BEGIN;
5	25241	alumno01	idle	Client	ClientRead	{}	SELECT pg_cancel_backend(25242);
6	25242	alumno01	idle	Client	ClientRead	{}	SELECT oid, pg_catalog.format_type(oid, NULL) AS typename FROM pg_catalog.pg_type WI

Preguntas para registrar:

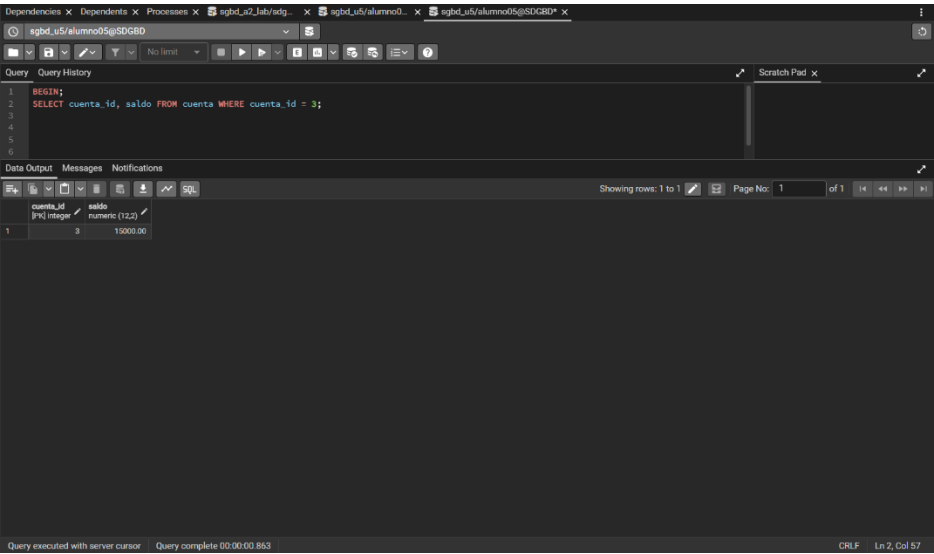
1. ¿Qué sesión está esperando?
2. ¿Que PID mantiene el bloqueo?
3. ¿La consulta está "lenta" o está esperando un recurso?
4. ¿Qué evidencia permite afirmarlo?

Segundo diagnóstico:

	pid integer	username name	state text	wait_event_type text	wait_event text	blocking_pids integer[]	query text
1	17850	alumno04	idle	Client	ClientRead	{}	/*pga4dash*/ SELECT 'session_stats' AS chart_name, pg_catalog.row_to_json(t) AS chart,
2	24192	alumno06	idle	Client	ClientRead	{}	/*pga4dash*/ SELECT 'session_stats' AS chart_name, pg_catalog.row_to_json(t) AS chart,
3	25239	alumno03	idle	Client	ClientRead	{}	SELECT pg_cancel_backend(25240);
4	25240	alumno03	idle in transacti...	Client	ClientRead	{}	BEGIN;
5	25241	alumno01	idle	Client	ClientRead	{}	SELECT pg_cancel_backend(25242);
6	25242	alumno01	idle	Client	ClientRead	{}	SELECT oid, pg_catalog.format_type(oid, NULL) AS typename FROM pg_catalog.pg_type WI
7	25246	alumno05	active	Lock	transaction...	{25240}	BEGIN;
8	25320	alumno06	idle	Client	ClientRead	{}	SELECT oid, pg_catalog.format_type(oid, NULL) AS typename FROM pg_catalog.pg_type WI
9	25364	alumno04	active	[null]	[null]	{}	SELECT pid, username, state, wait_event_type, wait_event, pg_blocking_pids()

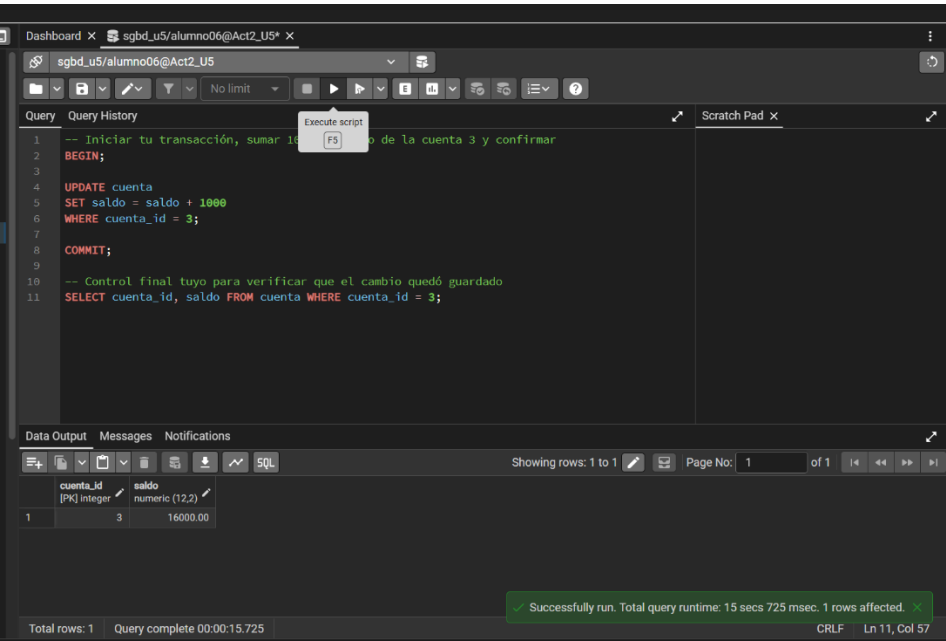
Experimento 2:

Sesión A:



Con ello ya hizo su primera lectura del saldo de la cuenta 3 y mantiene la transacción abierta, a continuación la sesión B:

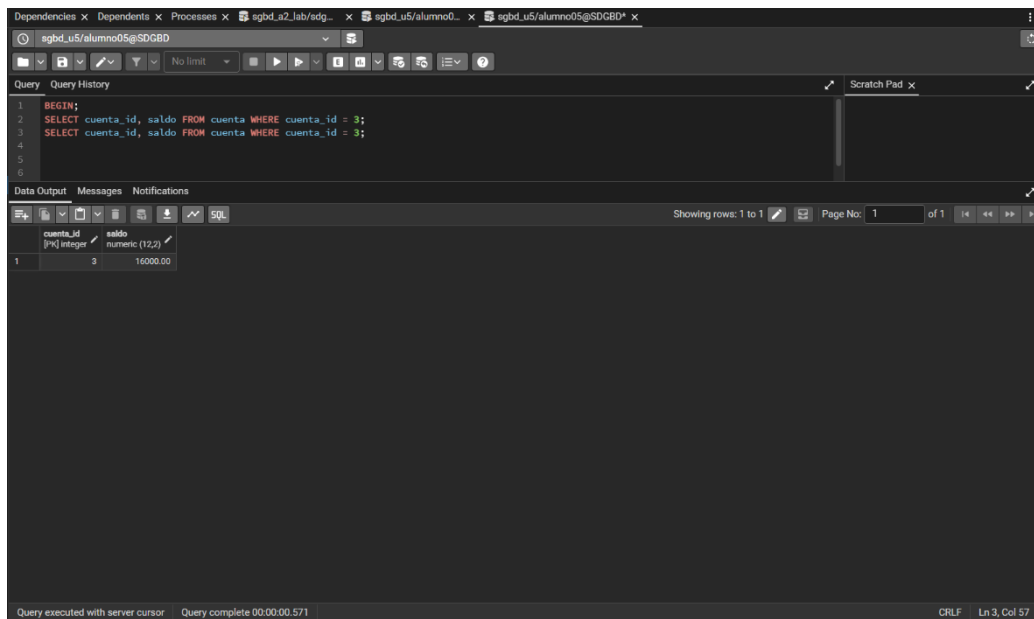
Sesión B:



El saldo de la cuenta 3 pasó de 15,000.00 a **16,000.00**, la transacción se confirmó con el **COMMIT** y la consulta de control final devolvió exactamente el valor esperado.

Segunda lectura de la sesión para ver si el motor le muestra el saldo viejo o el saldo nuevo a continuación:

Sesión A:



En su segunda lectura dentro de la misma transacción se puede ver el cambio de 1,000 que ya se había confirmado con el **COMMIT**.

Experimento 3:

Captura del paso 1A

- Sentencia:

```
BEGIN;  
  
UPDATE cuenta  
SET saldo = saldo - 10  
WHERE cuenta_id = 1;
```

- Resultado:

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 587 msec.

- UPDATE 1

Query returned successfully in 587 msec.

Captura del paso 1B

Query

Query History

1

BEGIN;

2

UPDATE cuenta SET saldo = saldo - 20 WHERE cuenta_id = 2;

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 319 msec.

- Paso 2

Query

Query History

1

UPDATE cuenta

2

SET saldo = saldo + 20

3

WHERE cuenta_id = 2;

Data Output

Messages

Notifications

Scratch Pad x

- Paso 3

Query Query History

```
1 UPDATE cuenta SET saldo = saldo + 10 WHERE cuenta_id = 1;
```

Data Output Messages Notifications

Waiting for the query to complete...

Total rows: Waiting for the query to complete... 00:01:36.812 CRLF Ln 1, Col 58

- Paso 4:

sgbd_u5/alumno01@U5A2* x

sgbd_u5/alumno01@U5A2

Query Query History

```
1 -- Paso 3: B intenta la cuenta 1, que retiene A. Aca se cierra el ciclo.
2 UPDATE cuenta SET saldo = saldo + 10 WHERE cuenta_id = 1;
```

Data Output Messages Notifications

ERROR: deadlock detected
 Process 27111 waits for ShareLock on transaction 2658; blocked by process 27107.
 Process 27107 waits for ShareLock on transaction 2659; blocked by process 27111.

SQL state: 40P01
 Detail: Process 27111 waits for ShareLock on transaction 2658; blocked by process 27107.
 Process 27107 waits for ShareLock on transaction 2659; blocked by process 27111.
 Hint: See server log for query details.
 Context: while updating tuple (0,44) in relation "cuenta"

- Diagnostico:

- Colgado:

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 8

Page No: 1

of 1

	pid integer	username name	state text	wait_event_type text	wait_event text	blocking_pids integer[]	query text
1	27035	alumno04	idle	Client	ClientRead	{}	/*pg4d\$*/ SELECT 'session_stats' AS chart_name, pg_catalog.row_to_json(t) AS chart_data FROM (\$
2	27036	alumno02	idle	Client	ClientRead	{}	SELECT CASE WHEN (SELECT count(extname) FROM pg_catalog.pg_extension WHERE extname='bdr
3	27096	alumno06	idle	Client	ClientRead	{}	/*pg4d\$*/ SELECT 'session_stats' AS chart_name, pg_catalog.row_to_json(t) AS chart_data FROM (\$
4	27107	alumno04	active	Lock	transaction...	{27111}	UPDATE cuenta
5	27110	alumno01	idle	Client	ClientRead	{}	SELECT case WHEN (SELECT count(extname) FROM pg_catalog.pg_extension WHERE extname='bdr
6	27111	alumno01	idle in transacti...	Client	ClientRead	{}	- Paso 1B: B bloquea primero la cuenta 2.
7	27128	alumno02	idle	Client	ClientRead	{}	SELECT oid, pg_catalog.format_type(oid, NULL) AS tyname FROM pg_catalog.pg_type WHERE oid = AN
8	27222	alumno01	active	[null]	[null]	{}	- Sesión bloqueada, sesión bloqueante y estado de espera. SELECT pid, username, state, wait...

Data Output		Messages		Notifications					
									
	pid integer 	locktype text 	relation regclass 	mode text 	granted boolean 				
1	27107	transaction...	[null]	ShareLock	false				
2	27107	relation	cuenta	RowExclusiveLo...	true				
3	27107	virtualxid	[null]	ExclusiveLock	true				
4	27107	relation	cuenta_pk...	RowExclusiveLo...	true				
5	27107	tuple	cuenta	ExclusiveLock	true				
6	27107	transaction...	[null]	ExclusiveLock	true				
7	27111	transaction...	[null]	ExclusiveLock	true				
8	27111	relation	cuenta_pk...	RowExclusiveLo...	true				
9	27111	relation	cuenta	RowExclusiveLo...	true				
10	27111	virtualxid	[null]	ExclusiveLock	true				
11	27222	relation	pg_locks	AccessShareLock	true				
12	27222	virtualxid	[null]	ExclusiveLock	true				

- Post deadlock

Data Output

Messages

Notifications

≡

+

SQL

	pid integer	locktype text	relation regclass	mode text	granted boolean
1	27107	relation	cuenta_pk...	RowExclusiveLo...	true
2	27107	relation	cuenta	RowExclusiveLo...	true
3	27107	virtualxid	[null]	ExclusiveLock	true
4	27107	transaction...	[null]	ExclusiveLock	true
5	27222	relation	pg_locks	AccessShareLock	true
6	27222	virtualxid	[null]	ExclusiveLock	true

Data Output

Messages

Notifications

≡

+

SQL

Showing rows: 1 to 8

Page No: 1

of 1

⏪

⏩

⏴

⏵

↺

↻

pid integer	username name	state text	wait_event_type text	wait_event text	blocking_pids integer[]	query text
1	27035 alumno04	idle	Client	ClientRead	0	/pgs4dash/ SELECT 'session_stats' AS chart_name, pg_catalog.row_to_json(t) AS chart_data
2	27036 alumno02	idle	Client	ClientRead	0	SELECT CASE WHEN (SELECT count(extname) FROM pg_catalog.pg_extension WHERE extname = 'pgs4dash')
3	27096 alumno06	idle	Client	ClientRead	0	/pgs4dash/ SELECT 'session_stats' AS chart_name, pg_catalog.row_to_json(t) AS chart_data
4	27107 alumno04	idle in transaction	Client	ClientRead	0	UPDATE cuenta
5	27110 alumno01	idle	Client	ClientRead	0	SELECT CASE WHEN (SELECT count(extname) FROM pg_catalog.pg_extension WHERE extname = 'pgs4dash')
6	27111 alumno01	idle in transaction (abort...	Client	ClientRead	0	- Paso 3: B intenta la cuenta 1, que retiene A. Aca se cierra el ciclo.
7	27128 alumno02	idle	Client	ClientRead	0	SELECT old, pg_catalog.format_type(old, NULL) AS typename FROM pg_catalog.pg_type WHERE
8	27222 alumno01	active	[null]	[null]	0	SET search_path TO sgbd_u5, public; -- Sesión bloqueada, sesión bloqueante y estado de esp